

# Memòria Pràctica I

---

## Coneixement, Raonament i Incertesa

Grup:

G08:30\_2

Membres:

Marc Sala

Àlex Pozo

Sofia Atienza - 1708916

## Índex

---

|                                               |     |
|-----------------------------------------------|-----|
| 1.- Introducció.....                          | pàg |
| 2.- Solució proposada.....                    | pàg |
| 2.1.- Exercici 1.....                         | pàg |
| 2.2.- Exercici 2.....                         | pàg |
| 2.3.- Exercici 3.....                         | pàg |
| 3.- Problemes trobats durant el projecte..... | pàg |
| 4.- Resultats obtinguts.....                  | pàg |
| 5.- Desenvolupament individual.....           | pàg |
| 6.- Conclusions individuals.....              | pàg |

## 1.- Introducció

---

El problema plantejat consisteix a resoldre de manera automàtica un tauler de mots encreuats. Donat que aquest es pot modelar com un CSP (Constraint Satisfaction Problem), la resolució consisteix a trobar una assignació de paraules d'un diccionari predefinit a les caselles d'un tauler, complint amb les condicions pròpies del joc. S'analitza també, la seva capacitat d'escalabilitat segons la mida del diccionari.

La pràctica es divideix en 3 parts, de menor a major complexitat:

- ❖ **Part C (Exercici 1):** Aborda el problema mitjançant l'algorisme de Backtracking pur sobre un tauler i com a recurs, s'utilitza un conjunt de diccionaris de mida reduïda com a base de la cerca.
- ❖ **Part B (Exercici 2):** Optimitza el procés anterior integrant la variant de Forward Checking, reduint l'arbre de cerca mitjançant el filtratge anticipat dels dominis de les variables no assignades.
- ❖ **Part A (Exercici 3):** Posa a prova l'escalabilitat i l'eficiència del codi aplicant la solució de la part B a un escenari a gran escala amb un diccionari de més de 580.000 paraules.

Aquesta pràctica té com a objectiu principal assimilar els conceptes de cerca amb satisfacció de restriccions (CSP) i aplicar de forma correcta i funcional tan l'algorisme Backtracking (BT) com la seva variant amb Forward Checking (BT-FC). Es diferencien en com i quan es detecten conflictes amb les variables futures no assignades, aprofundirem més sobre aquest aspecte al llarg del desenvolupament de la memòria.

## 2.- Solució proposada

---

### 2.1.- Part C (Exercici 1):

Per a aquesta primera part disposem dels següents arxius:

- 1) Un fitxer anomenat *diccionari\_CB\_v3.txt* on hi ha 107 paraules diferents
- 2) Un document on hi ha l'esquema d'un tauler de mots encreuats *crossword\_CB\_v3.txt*

A continuació, s'especifiquen les característiques del problema:

- **Variables:**

En aquest cas, les variables són els diferents espais contigus (horitzontals i verticals) on s'ha d'ubicar una mot dins del tauler del joc.

En el codi, definim les variables segons cada element de la llista *tasques*:

La tupla  $(H/V, x, allargada, y)$  conforme una variable.

- $H$  o  $V$  indica l'orientació (horitzontal o vertical).
- $x$  i  $y$  són les coordenades inicials de la primera casella.
- *allargada* és el nombre de lletres que caben en el forat.

Si el tauler conté  $N$  caselles, tindrem  $N$  variables:  $X = \{X_1, X_2 \dots X_N\}$ .

- **Domini de les variables:**

El domini  $D_i$  d'una variable  $X$  és el conjunt de paraules del diccionari que tenen exactament la mateixa longitud que el forat associat a  $X$ .

$D_i = \{p \text{ que pertanyen a Diccionari} \mid \text{len}(p) = L_i\}$

En el codi, l'ús de l'estructura *dicP* fa que el domini d'una variable de longitud  $L_i$  vingui donat directament per la llista *dicP*[ $L_i$ ].

- **Restriccions del problema:**

El problema presenta 3 restriccions:

- Restricció global de No Repetició: En primer lloc, una paraula no es pot utilitzar més d'una vegada. En el codi això es comprova mitjançant la condició: *p not in visitats* (funció de *backtracking* i funció de *actualitzarDominis*).

- Restricció Binària de Creuament: Quan dues paraules en diferents posicions (una en vertical i l'altre en horitzontal) es creuen, el caràcter en la posició de creuament ha de ser idèntic per als dos mots. En el codi, es comprova a les funcions *posarParaulaH* i *posarParaulaV*: una casella només s'accepta si conté un '0' (està buida) o si ja conté exactament el mateix caràcter (`matriu[x][y] == char`).
- Restricció Unària (mida): la mida de la paraula assignada *p* ha de coincidir exactament amb la longitud del espai disponible.

- **Tamany de l'espai de solucions inicial:**

Inicialment el codi prefiltra les paraules segons la mida del forat abans de començar la cerca, l'espai de solucions inicial que l'arbre de cerca explora, ve donat pel producte de les mides dels dominis de cada variable:

$$\text{Espai Total} = |D_i|(i=1 - N) = |D\_1| \times |D\_2| \times |D\_3| \dots |D\_N|$$

Si no s'hagués filtrat inicialment per longitud o unicitat, l'espai total seria

$$\text{Espai Total} = M^N = 107^N, \text{ sent } M \text{ el diccionari amb les } 107 \text{ paraules.}$$

- **Estratègies utilitzades per reduir la profunditat mitjana de les branques:**

- Filtratge previ per longitud (*dicP*):

En lloc de provar totes les 107 paraules del diccionari en cada iteració, **es redueix el factor de ramificació (branching factor) filtrant el domini per la mida del forat (*dicP[allargada]*). Això evita explorar branques que fallarien a la primera comprovació de longitud. //falta**

- Poda immediata (Pruning):

A les funcions *posarParaulaH* i *posarParaulaV*, en el moment en què un caràcter de la paraula no coincideix amb la lletra ja existent al tauler, la funció retorna False. Això fa que es talli la branca en aquell precís moment sense fer la crida recursiva.

En aquesta part (Part C) no s'ha aplicat encara cap estratègia més avançada com Forward Checking, ja que es demana a la següent part. És per això que l'ordre d'exploració de les variables ve donat per l'ordre de la llista *tasques*.

2.1.- Part B (Exercici 2):

2.1.- Part A (Exercici 3):

## Memòria

La memòria ha de contenir com a mínim els següents apartats..

- Portada
- Índex
- Introducció: explicació del problema
- Solució proposada: com heu resolt el problema, les estructures de dades utilitzades, etc
- Problemes trobats durant el projecte i com els heu resolt
- Resultats
- Desenvolupament individual (cada membre explica la seva

contribució)

- Conclusions individuals (cada membre extreu les seves pròpies)

A l'informe cal que contesteu aquestes preguntes, a més a més d'explicar com heu resolt el problema:

### 3.- Problemes trobats durant el projecte

---